

Tarefa 2 — Dependências entre instruções e o efeito da otimização do compilador

Objetivo

Observar como a **dependência entre uma iteração e a próxima** afeta o tempo de execução de um laço, e entender o que acontece quando o compilador é autorizado a otimizar o código.

O que foi implementado

Dois laços operando sobre um vetor de 100 milhões de números inteiros:

Laço simples — soma todos os elementos em uma única variável:

```
long long soma = 0;
for (int i = 0; i < N; i++)
    soma += A[i];
```

Laço com múltiplas variáveis — soma os elementos em variáveis independentes:

```
// exemplo com 4 variáveis
long long soma1 = 0, soma2 = 0, soma3 = 0, soma4 = 0;
for (int i = 0; i < N; i += 4) {
    soma1 += A[i];
    soma2 += A[i + 1];
    soma3 += A[i + 2];
    soma4 += A[i + 3];
}
```

Variantes com **2, 4, 8 e 12 variáveis** acumuladoras foram testadas.

Cada versão foi compilada de três formas: **sem otimização (-O0)**, com **otimização moderada (-O2)** e com **otimização máxima (-O3)**.

Resultados

Laço	Variáveis	-O0 (s)	-O2 (s)	-O3 (s)
laco_soma	1	0.1764	0.0225	0.0256
laco_unroll_2	2	0.0863	0.0278	0.0272
laco_unroll_4	4	0.0514	0.0274	0.0279

Laço	Variáveis	-O0 (s)	-O2 (s)	-O3 (s)
laco_unroll_8	8	0.0504	0.0223	0.0277
laco_unroll_12	12	0.0455	0.0276	0.0277

Análise

Sem otimização (-O0): por que uma variável é mais lenta

No laço simples, cada soma depende do resultado da soma anterior: para calcular **soma** na iteração 5, o processador precisa que a iteração 4 já tenha terminado. Não tem como fazer as duas ao mesmo tempo — é uma fila de espera obrigatória.

O processador consegue executar várias operações simultaneamente quando elas são independentes. Mas quando uma operação precisa esperar o resultado da anterior, essa execução em paralelo trava. Com 100 milhões de iterações em fila, o tempo acumula: **0.1764s**.

O laço com múltiplas variáveis resolve isso: **soma1** e **soma2** nunca dependem uma da outra, então o processador pode calculá-las ao mesmo tempo. Resultado:

Variáveis	Tempo -O0 (s)	Ganho em relação ao laço simples
1 (laco_soma)	0.1764	referência
2	0.0863	2.0× mais rápido
4	0.0514	3.4× mais rápido
8	0.0504	3.5× mais rápido
12	0.0455	3.9× mais rápido

O ganho cresce até 12 variáveis e depois começa a regredir. Isso acontece porque o processador tem uma quantidade limitada de espaço interno para guardar variáveis ativas simultaneamente. Com muitas variáveis sem otimização, esse espaço se esgota e o programa passa a usar a memória como rascunho extra, o que é mais lento.

Com otimização (-O2 e -O3): o compilador faz o trabalho

Com otimização ativada, todos os laços ficam com tempos parecidos (~0.022–0.028s). O compilador identifica que o laço simples é uma soma de todos os elementos e transforma o código automaticamente para executar várias somas ao mesmo tempo — exatamente o que o laço com múltiplas variáveis fazia manualmente.

O resultado mais expressivo é o do próprio laço simples: sem otimização ele levava 0.1764s, com otimização passou para 0.0225s — **8 vezes mais rápido**, sem mudar uma linha do código-fonte.

Os laços com múltiplas variáveis também melhoram com otimização, mas menos (~3×), porque já estavam mais próximos do ideal.

Resumo dos ganhos

O que comparamos	Resultado
laco_soma sem vs. com otimização	8× mais rápido — compilador resolve a dependência
laco_soma vs. laco_unroll_2 sem otimização	2× mais rápido — divisão básica do trabalho
laco_soma vs. laco_unroll_12 sem otimização	3.9× mais rápido — melhor resultado manual
laco_soma vs. laco_unroll_12 com otimização	igual — compilador chega no mesmo lugar

Conclusão

Quando não há otimização, dependências entre iterações criam gargalos reais e visíveis — o mesmo laço rodando quase 4× mais lento só por usar uma variável em vez de várias. Com otimização ativada, o compilador identifica esse padrão e resolve sozinho, igualando o desempenho de todos os laços.

Isso mostra que **compilar com otimização** é sempre o primeiro passo, e que mudanças manuais no código só fazem sentido quando há evidência de que o compilador não conseguiu otimizar por conta própria.

Código

laco_soma_simples.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 100000000

static int A[N];

int main() {
    // laco 1: inicializacao do vetor
    for (int i = 0; i < N; i++)
        A[i] = i + 2;

    // laco 2: soma acumulativa com dependencia entre iteracoes
    long long soma = 0;

    double start = omp_get_wtime();

    for (int i = 0; i < N; i++)
        soma += A[i];

    double end = omp_get_wtime();

    printf("%.6f\n", end - start);
    printf("Soma: %lld\n", soma);

    return 0;
}
```

laco_unroll_2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 100000000

static int A[N];

int main() {
    for (int i = 0; i < N; i++)
        A[i] = i + 2;

    long long soma1 = 0, soma2 = 0;

    double start = omp_get_wtime();

    for (int i = 0; i < N; i += 2) {
        soma1 += A[i];
        soma2 += A[i + 1];
    }

    double end = omp_get_wtime();

    printf("%.6f\n", end - start);
    printf("Soma: %lld\n", soma1 + soma2);

    return 0;
}
```

laco_unroll_4.c

```
#include <stdio.h>
#include <omp.h>

#define N 100000000

static int A[N];

int main() {
    for (int i = 0; i < N; i++)
        A[i] = i + 2;

    long long soma1 = 0, soma2 = 0, soma3 = 0, soma4 = 0;

    double start = omp_get_wtime();

    for (int i = 0; i < N; i += 4) {
        soma1 += A[i];
        soma2 += A[i + 1];
        soma3 += A[i + 2];
        soma4 += A[i + 3];
    }

    double end = omp_get_wtime();

    printf("%.6f\n", end - start);
    printf("Soma: %lld\n", soma1 + soma2 + soma3 + soma4);

    return 0;
}
```

laco_unroll_8.c

```
#include <stdio.h>
#include <omp.h>

#define N 100000000

static int A[N];

int main() {
    for (int i = 0; i < N; i++)
        A[i] = i + 2;

    long long soma1 = 0, soma2 = 0, soma3 = 0, soma4 = 0;
    long long soma5 = 0, soma6 = 0, soma7 = 0, soma8 = 0;

    double start = omp_get_wtime();

    for (int i = 0; i < N; i += 8) {
        soma1 += A[i];
        soma2 += A[i + 1];
        soma3 += A[i + 2];
        soma4 += A[i + 3];
        soma5 += A[i + 4];
        soma6 += A[i + 5];
        soma7 += A[i + 6];
        soma8 += A[i + 7];
    }

    double end = omp_get_wtime();

    printf("%.6f\n", end - start);
    printf("Soma: %lld\n", soma1 + soma2 + soma3 + soma4 +
        soma5 + soma6 + soma7 + soma8);

    return 0;
}
```

laco_unroll_12.c

```
#include <stdio.h>
#include <omp.h>

#define N 100000008 // multiplo de 12

static int A[N];

int main() {
    for (int i = 0; i < N; i++)
        A[i] = i + 2;

    long long soma1 = 0, soma2 = 0, soma3 = 0, soma4 = 0;
    long long soma5 = 0, soma6 = 0, soma7 = 0, soma8 = 0;
    long long soma9 = 0, soma10 = 0, soma11 = 0, soma12 = 0;

    double start = omp_get_wtime();

    for (int i = 0; i < N; i += 12) {
        soma1 += A[i];
        soma2 += A[i + 1];
        soma3 += A[i + 2];
        soma4 += A[i + 3];
        soma5 += A[i + 4];
        soma6 += A[i + 5];
        soma7 += A[i + 6];
        soma8 += A[i + 7];
        soma9 += A[i + 8];
        soma10 += A[i + 9];
        soma11 += A[i + 10];
        soma12 += A[i + 11];
    }

    double end = omp_get_wtime();

    printf("%.6f\n", end - start);
    printf("Soma: %lld\n", soma1 + soma2 + soma3 + soma4 +
        soma5 + soma6 + soma7 + soma8 +
        soma9 + soma10 + soma11 + soma12);

    return 0;
}
```


Compilação e execução

```
gcc -O0 -fopenmp laco_soma_simples.c -o laco_soma_00 -lm
gcc -O2 -fopenmp laco_soma_simples.c -o laco_soma_02 -lm
gcc -O3 -fopenmp laco_soma_simples.c -o laco_soma_03 -lm

./laco_soma_00
./laco_soma_02
./laco_soma_03
```